

From Search Scaling to Learning Scaling: Task-Time Search, Runtime Continual Learning, and Recursive Improvement

Ying Wen

Shanghai Jiao Tong University
Shanghai Innovation Institute

ying.wen@sjtu.edu.cn

August 24, 2026

Abstract

Improved task performance can result from additional computation within a fixed learner or from experience that persistently changes the learner used for later decisions. This position paper distinguishes Search Scaling from Learning Scaling by declaring the changing resource, the persistent learning state, and the future operating conditions held fixed. It develops this distinction through harness-induced search constraints, residual sources of error, first-person experience, and Alberta-style continual reinforcement learning. Recursive computation, recursive optimization, and recursive learning are analyzed as different processes, including their roles in AI-for-AI research. An evaluation protocol combines reset tests, fixed future computation, retention, late-stage plasticity, and learning-efficiency measurements. The proposed response relations and conceptual figures are analytical definitions and research proposals, not established universal scaling laws.

Keywords: search-scaling, learning-scaling, continual-rl, llm-agents, harness, bias-variance

Search Scaling changes how much computation an already-formed learner devotes to the current problem. Learning Scaling changes how accumulated experience continually changes that learner so that it performs better later under the same computational conditions—and remains able to learn.

The limits of my language mean the limits of my world. — Ludwig Wittgenstein, *Tractatus Logico-Philosophicus*, §5.6

Wittgenstein was not writing about machine learning. Yet the sentence gives search an unusually precise starting point. For a search system, its “language” includes the states it can represent, the actions it can propose, the consequences it can predict, and the outcomes its evaluator recognizes as good. More search can explore that language more thoroughly. It cannot, by itself, express distinctions that are absent from the representation, action set, or evaluation process.

Consider autonomous driving. A typical system uses recorded road experience together with synthetic data produced by simulators. Recorded experience preserves traffic interactions that have already occurred. Simulation can cheaply generate rare weather, traffic, sensor, and road conditions and provide explicit training feedback. Pretraining and RL post-training turn both into state representations, predictions of action consequences, driving policies, and approximate evaluators. A harness connects the model to sensors, controls, safety constraints, trajectory generation, and commitment rules.

Once the car encounters roadworks, it may compare more braking, waiting, lane-change, and detour trajectories. If the model and harness remain fixed, this is **Search Scaling**. More computation can reduce the chance that a good reachable trajectory is simply missed. It cannot guarantee that an unmodeled change in friction, a new sensor failure, or an unfamiliar pattern of human behavior suddenly enters the system's state representation.

Actual road interaction therefore continues to produce conditions absent from the training distribution and simulator. The usual remedy is periodic centralized iteration: engineers inspect failures, revise the simulator and training data, and release another shared model. This can improve the fleet. It also leaves an adaptation interval between two training runs, and it does not imply that a deployed agent has learned from the consequences of its own actions.

Learning Scaling concerns this second relation. The agent receives feedback along a first-person, action-conditioned stream of experience. Persistent updates alter later prediction and control. With future task-time computation held fixed, long-run control improves with accumulated experience while retention and late-stage plasticity remain intact. This is not an argument for unconstrained online weight updates in safety-critical systems. It is an argument that the ability to keep learning from real consequences is a separate scientific problem.

Central distinction. The dominant scaling path in current AI has enlarged search over progressively higher-level candidates: outputs, reasoning trajectories, actions, harnesses, code, algorithms, and AI designs. Search Scaling covers more combinations inside a given representation, proposal, learning-target, and evaluation system. Learning Scaling changes the learner that will represent, evaluate, formulate intermediate learning targets, and update from future experience. Alberta-style continual RL gives this distinction a strong form: task parameters, per-parameter step sizes and other meta-parameters, and feature, state, and abstraction construction evolve together; the agent must also construct what to learn next from its capability and experience. If that process makes the agent learn faster later in its run, it becomes a candidate mechanism for runtime RSI.

Search Scaling holds the initial learning state and search procedure fixed, then studies current-task or current-design performance as search computation changes.

Learning Scaling specifies a long-run objective, an experience-generating process, and future operating conditions, then studies how continuing control and continued learnability—including the construction and revision of intermediate learning targets—change with accumulated first-person experience.

A reset test makes the difference concrete. Clear the task-local context, candidate trajectories, search tree, and temporary cache. Restore the same computation for the next decision. If the advantage disappears, it lived in the previous search trajectory. If it remains under equal future computation, it may reside in persistent learning state.

The word “to” in the title adds a research axis; it does not announce a historical stage or a replacement. Sutton's *Bitter Lesson* [1] treats search and learning as two general methods that can exploit computation. The relevant distinction is therefore not search versus learning as competing ideologies. It is **using more computation to select a better candidate now versus using experience to change what can be predicted, proposed, valued, and learned later**.

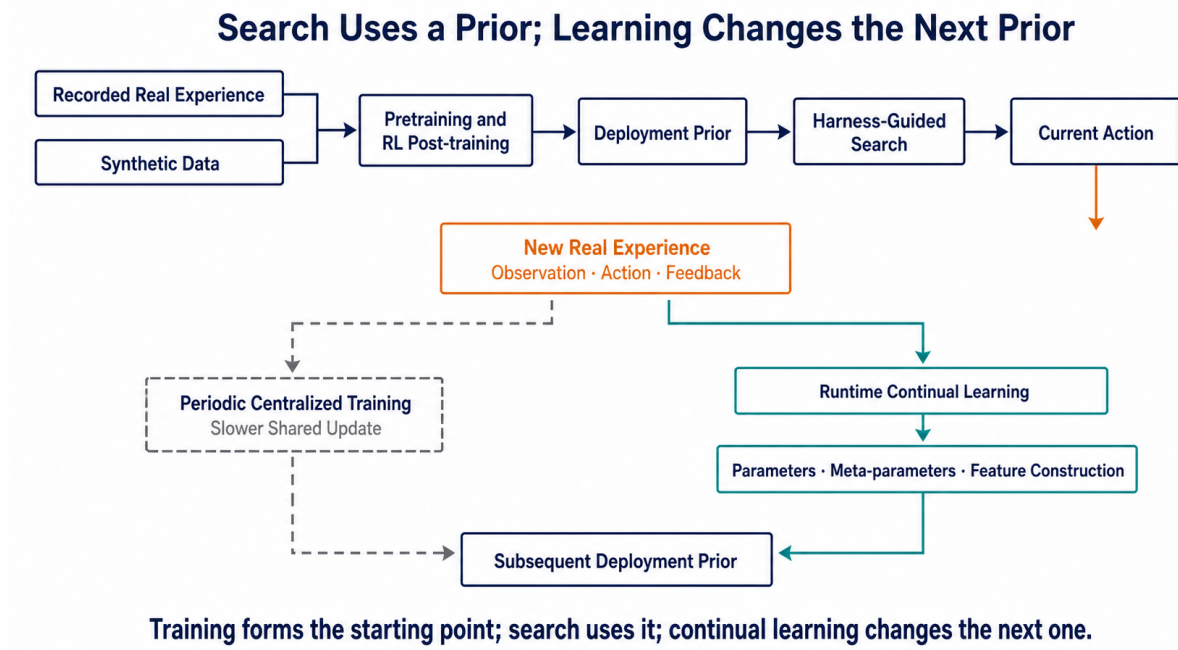


Figure 1. Recorded real experience and synthetic data form a deployment prior through pretraining and RL post-training. Deployment produces new real experience, which can enter periodic centralized training or runtime continual learning that changes parameters, meta-parameters, and feature construction for later decisions.

1 One improvement in performance, two capability-forming processes

A scaling curve is incomplete until it states where the changing resource is used. Does computation expand candidates for the current problem, or does experience produce a persistent update that affects later problems? The comparison must also state the starting state of independent runs, the task distribution, the reset conditions, and the operating conditions that remain fixed.

Both relations can be written within one continuing interaction process. Let the agent’s first-person history at time t be

$$h_t = (o_0, a_0, r_1, o_1, \dots, a_{t-1}, r_t, o_t),$$

where o_t , a_t , and r_{t+1} denote an observation, an action, and the reward or environmental feedback following that action. Let

$$\Lambda_t = (\theta_t, \eta_t, \phi_t, \Gamma_t)$$

denote the learning state that can continue to affect later decisions and updates. Task parameters θ_t carry predictions, value estimates, environment models, and control policies; meta-parameters η_t regulate per-parameter step sizes and update dynamics; ϕ_t constructs features, agent state, and state–behavior abstractions from history; and Γ_t constructs intermediate learning targets from current capability and experience. The long-run control objective J remains externally specified. Γ_t may select which predictions, opponents, behavioral distinctions, or environmental variations to learn next only in service of that objective and its constraints.

A current decision can then be written as

$$a_t = \mathcal{S}(\phi_t(h_t); \Lambda_t, \mathcal{H}, c_t, \omega_t),$$

where $\mathcal{S}$ is a search or planning procedure, $\mathcal{H}$ is the harness, c_t is task-time search computation, and ω_t collects randomness from proposal sampling, tool feedback, and branch ordering. Search may create a changing tree, context, or candidate set within the task; these transient quantities do not thereby become cross-task learning state. Interaction produces

$$\xi_{t+1} = (o_t, a_t, r_{t+1}, o_{t+1}), \quad g_t = \Gamma_t(h_t), \quad \Lambda_{t+1} = \mathcal{U}(\Lambda_t, \xi_{t+1}; g_t),$$

where $\mathcal{U}$ is the update operator that writes experience into later prediction, control, and learning. For one Search Scaling curve, Λ and $\mathcal{H}$ are held fixed. If experience persistently updates the harness, its persistent part belongs to learning state in the Learning Scaling analysis.

1.1 Search Scaling

Call model parameters, long-term memory, reusable policy structure, and update rules the **learning state**. Call the state representation, available actions, candidate expansion, evaluation, compute allocation, stopping, and submission rules the **search procedure**.

Search Scaling begins from replicated systems with the same initial learning state. It preserves the search procedure and task distribution while varying task-time computation. Computation may be counted in generated tokens, candidates, tree nodes, tool calls, simulated environment steps, GPU-seconds, or wall-clock time. A single curve must use a consistent unit.

If X is drawn from a fixed task distribution and J_{task} evaluates the submitted result, the conditional response at fixed learning state is

$$F_S(c \mid \Lambda) = \mathbb{E}_{X, \omega} [J_{\text{task}}(\mathcal{S}(X; \Lambda, \mathcal{H}, c, \omega))].$$

Search Scaling changes c . The contrast

$$\Delta_S = F_S(c_2 \mid \Lambda) - F_S(c_1 \mid \Lambda)$$

asks what happens when the same learner devotes more search computation to the current problem. It does not imply that Λ has changed or that structure found during search will affect later problems.

In autonomous driving, the learned road representation, action-consequence predictor, and control prior belong to learning state. Sensor processing, executable maneuvers, trajectory roll-out, safety evaluation, and commitment rules belong to the search procedure. The amount of trajectory comparison before an action is the changing resource.

Task-local state may evolve within a search run. A search tree, intermediate draft, or candidate archive need not be frozen node by node. “Fixed learning state” means that independent repetitions start from the same persistent state and use the same expansion, evaluation, allocation, stopping, and submission rules. If a chosen candidate changes the state used by later independent tasks, that cross-task update must be reported separately.

1.2 Learning Scaling

In continual reinforcement learning, experience is not an externally sampled dataset. At time t , the agent acts from its history; the action changes the next observation and feedback; learning changes the policy that produces later experience. Accumulated experience therefore grows along a first-person stream jointly induced by policy and environment.

Where task boundaries exist, task-local context, search trees, and temporary caches can be cleared at several experience checkpoints. Future tasks are then evaluated with the same task-time computation. In continuing control, where no natural reset exists, a long-run objective

must be stated first: average reward, discounted return, constrained utility, or another domain-specific criterion. The question is whether additional experience leads to better later control, faster recovery after change, or more efficient learning of unfamiliar structure.

Let the learning state after e steps of first-person experience be

$$\Lambda_e = \mathcal{U}(\dots \mathcal{U}(\mathcal{U}(\Lambda_0, \xi_1), \xi_2) \dots, \xi_e).$$

With future search computation fixed at c^* , Learning Scaling studies

$$F_L(e \mid c^*) = \mathbb{E}[J_{\text{future}}(\Lambda_e, c^*)], \quad \Delta_L = F_L(e_2 \mid c^*) - F_L(e_1 \mid c^*).$$

The formal distinction is therefore direct: Search Scaling holds Λ fixed and changes c ; Learning Scaling fixes future c^* and lets accumulated experience change Λ_e through $\mathcal{U}$. Because policy changes with Λ_e , independent runs generally produce different action-conditioned experience streams. Feeding the same recorded sequence to multiple systems can isolate how update rules use that record, but cannot replace the continuing interaction process.

Persistent change alone is not enough. Learning Scaling must examine long-run task utility, recovery, transfer, retention, abstraction formation and revision, late-stage plasticity, and new-task learning efficiency. Better retrieval on repeated items shows reuse, but does not by itself establish transferable or continuing learning.

Dimension	Search Scaling	Learning Scaling
Independent variable	Search computation for the current task or design round	Accumulated first-person experience
Held fixed	Initial learning state, search procedure, task distribution	Long-run objective, experience process, future task-time computation and evaluation
After a task	Task-local search state is reset for an independent repetition	Declared updates persist into later interaction
Main response	Current-task performance or design-submission quality	Continuing control, recovery, transfer, retention, abstraction formation and revision, plasticity, and later learning efficiency
Typical confusion	Treating more candidates as learning	Treating any write, parameter change, or remembered item as continual learning

Two Different Objects of Scaling

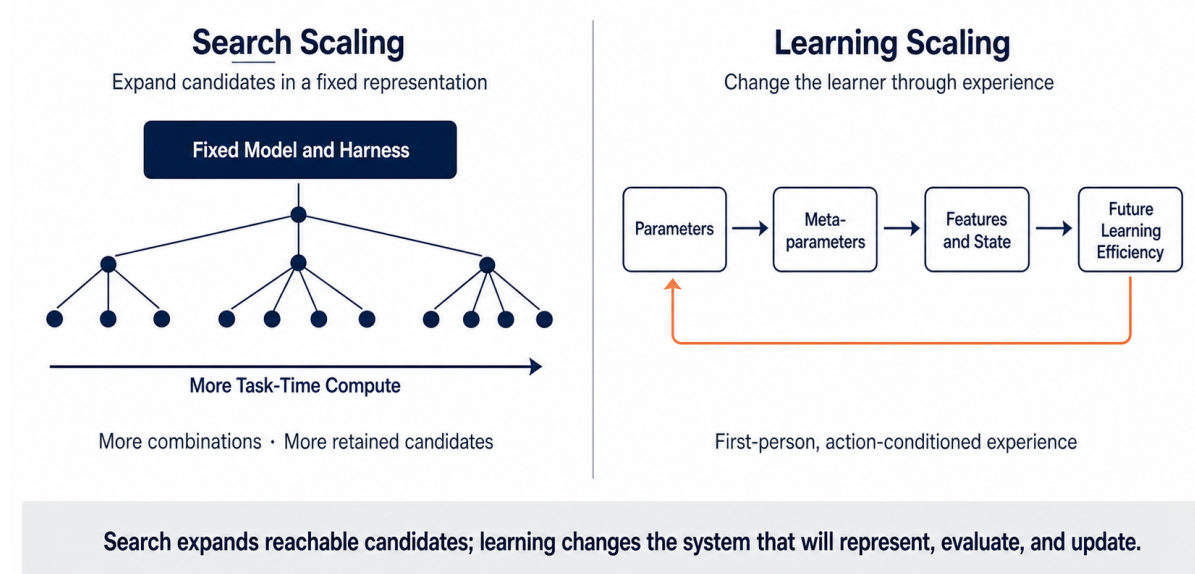


Figure 2. Search Scaling expands reachable candidates under a fixed model and harness. Learning Scaling lets parameters, meta-parameters, features, and state construction evolve with experience, and is judged by continuing control and later learning efficiency.

2 Recorded real experience, synthetic data, and new experience after deployment

Autonomous driving places the modern AI pipeline on one page. Recorded real experience enters centralized training; synthetic data extend conditions that designers know how to generate and evaluate. Pretraining and RL post-training write statistical structure into shared model state. A harness connects the model to a concrete environment. Search Scaling expands candidates for the present decision. Runtime continual learning uses new experience produced by actual actions to alter later decisions.

SMARTS [2] treats interaction among diverse road users as a central problem for autonomous-driving reinforcement learning and provides a scalable simulation platform for multi-agent research. It illustrates the scientific value of simulation without implying that a simulator exhausts future road interaction.

These are complementary, not mutually exclusive, mechanisms. Recorded experience and synthetic data are centralized training inputs. Pretraining and RL post-training are model-construction processes. Search Scaling is computation with learning state fixed. Learning Scaling is a relation between accumulated action-conditioned experience and later control and learnability.

Component	Signal source	What it forms or changes	What it does not solve alone
Recorded real experience	Recorded users, devices, text, images, and environment interactions	A reference drawn from processes that actually occurred	Unexecuted counterfactual actions; conditions induced by future policies

Component	Signal source	What it forms or changes	What it does not solve alone
Synthetic data	Simulators, generative models, self-play, programs, or formal rules	Rare conditions, adjustable difficulty, inexpensive labels	Factors omitted by the generator and evaluator
Pretraining	Predictive objectives over large real and synthetic corpora	Shared representation, candidate generation, consequence prediction, initial value estimates	Residual mismatch with deployment distributions and long-run objectives
RL post-training	Policy trajectories, rewards, preferences, verifiable checks, environment feedback	Reasoning, tool use, action, recovery, stopping, and evaluation tendencies	New regularities and path-dependent experience after training stops
Harness and Search Scaling	Current inputs, tools, constraints, and task-time evaluation	A restricted search graph and computation allocation within the fixed prior	Universally removing unreachable actions, missing state, or systematic evaluation error
Offline model iteration	Aggregated and filtered records from many deployments	Periodic improvement of shared capability	Adaptation delay, private experience, local change, and current-policy interaction
Runtime continual learning	The deployed agent's own observations, actions, and consequences	Persistent correction of later prediction, control, and learning	Safety, stability, retention, and plasticity without explicit mechanisms

2.1 Recorded real experience describes the past, not a complete policy-dependent future

Recorded data describe what has happened under earlier collection policies. They can include human behavior, previous agents, or earlier runs of the same system. Once sampled into centralized training, however, the record becomes an external training input. It does not contain the consequences of every action that a newly deployed policy might take.

The gap is not merely missing volume. A new policy changes the distribution of states it visits. Some low-frequency conditions were never recorded. Some user, organization, and device-specific information cannot be centralized. Real data are indispensable, but a historical record cannot fully specify a policy-dependent future.

2.2 Synthetic data are useful when generation and evaluation admit independent correction

“Synthetic data” covers very different mechanisms. Self-play in a fully specified game can use real terminal outcomes. Program execution and proof checking can reject many invalid candidates. Domain randomization can vary parameters already represented by a simulator. Generative models can produce text, images, or tasks at very low marginal cost.

AlphaZero [3] shows how self-play can create valuable trajectories when rules and outcomes are complete. Tobin et al. [4] demonstrated sim-to-real transfer through domain randomization in a specific robotic vision and grasping setting. These successes do not imply that arbitrary generators capture reality. They show that synthetic data are strongest when variation is tied to a well-specified environment or an external check.

Sutton and Javed [5] identify a boundary narrower than a rejection of synthetic data as such. Their criticism concerns synthetic data treated as a general substitute for continuing experience in an open world: under the Big World premise, every finite human-designed simulator expresses only a small part of the world. Javed also acknowledges the value of simulation and argues for models learned from an agent’s own experience and corrected through continual learning. The issue is therefore whether synthetic generation can receive independent correction and whether the deployed agent can continue learning from new experience.

Silver and Sutton’s *Era of Experience* [6] places the same distinction inside agent–environment interaction. A fixed synthetic procedure will eventually be outstripped as an agent becomes stronger; scalable experience is instead generated by actions, environmental consequences, and rewards grounded in those consequences. The relevant division is therefore not a static real-versus-synthetic label, but whether experience generation changes with the policy, receives correction from external consequences, and enters later learning.

The opposite failure is recursive self-reference without adequate correction. Shumailov et al. [7] study later generative models repeatedly trained on material produced by earlier models and report loss of tail information and model collapse in the processes they analyze. This does not make all synthetic data harmful. It identifies a condition under which the generator and learner share an error and no independent signal repairs it.

The right distinction is therefore not real good, synthetic bad. It is whether generation covers task-relevant factors and whether evaluation, real interaction, or formal structure supplies an independent correction.

2.3 Pretraining and RL post-training are learning, but they set a deployment starting point

Pretraining is genuine learning. Scaling data, parameters, and training computation changes model parameters and often improves broad downstream performance. The response studied by Kaplan et al. [8] and Hoffmann et al. [9] is a training-scaling relation: training resources versus properties of the resulting trained model.

From the deployment perspective, the key temporal fact is that the update largely happens before use. The trained model begins deployment with a fixed shared prior. Residual mismatch remains for at least three reasons: collection coverage is incomplete; predictive or preference objectives differ from long-run control utility; and future experience depends on the actions of the deployed policy.

RL post-training also changes model parameters during construction. Reasoning RL can shape decomposition, verification, recovery, and stopping. Agentic RL extends trajectories through tools and environment actions. POAD [10], for example, refines language-agent credit assignment from complete actions to intra-action tokens. Meta-RL can train an adaptation procedure. At deployment, however, a fixed post-trained policy may simply unfold longer reasoning or action trajectories. More RL training computation and more task-time search computation are different independent variables.

The issue is not that pretraining and post-training “do not learn.” They clearly do. The issue is that a powerful prior built at one time is not the same process as an agent that continues to learn from its own consequences at another time.

2.4 Offline releases and runtime learning operate at different time constants

Periodic retraining is well suited to common structure aggregated across many users and tasks. It is weaker when the relevant regularity is local, private, rapidly changing, or induced by the current agent’s actions. Between releases, a frozen system can search more, store information

in task context, or retrieve prior records. If those changes are cleared, the policy begins the next task from the same persistent state.

Runtime learning does not have to mean immediate unconstrained weight modification. Persistent state may include carefully governed parameters, value estimates, predictive state, long-term memory, reusable policy structure, or parts of the harness. The scientific requirement is narrower: later behavior must change because experience produced a persistent update, and the update must be assessed through later control and learnability rather than through the fact that something was written.

3 Why an LLM can be a general searcher—and why a harness is still necessary

Search computation is useful only when it is directed. In an intractably large combinatorial space, a search procedure determines how states are represented, what operations are legal, where expansion begins, how intermediate candidates are evaluated, how computation is allocated, and when a result is committed.

3.1 The learned sources of general search

“General” does not mean uniformly strong on every problem. It means that one parameterized generator can condition on many task descriptions, contexts, and feedback signals, then produce candidate sequences across many symbolic and action domains.

Pretraining gives the model a broad conditional distribution over continuations. Autoregressive generation assigns probabilities to next tokens, tool calls, or actions. Those learned statistics enter search in at least three roles:

1. **Consequence prediction.** Given a state and action, the model can predict later states, tool returns, or language feedback, serving as an approximate environment-consequence predictor.
2. **Candidate generation and policy.** The model can propose an action or complete trajectory and concentrate computation on selected branches.
3. **Value estimation and evaluation.** The model can compare intermediate states, estimate success, or rank candidate quality.

Autoregressive generation also defines an implicit prefix tree. A prefix is a node; a token, tool call, or environment action is an edge. Sampling expands branches, sequential revision follows feedback, and a verifiable checker or environmental return ranks and prunes candidates. Large Language Monkeys [11] reports broader candidate coverage from repeated sampling on several task classes. Snell et al. [12] show that useful test-time computation depends on model capability, task difficulty, and allocation between parallel and sequential search. TS-LLM [13] uses a learned value function to guide language-model tree search, while ToolPRM [14] uses fine-grained process evaluation to guide beam search over structured function calls.

An LLM is therefore a learned, amortized source of proposals, predictions, and approximate values. It is not, by itself, a complete search algorithm for an external task.

3.2 A harness formulates and controls the search problem

The model does not independently define external state, executable actions, permissions, tool semantics, rollback, or commitment. A **harness** supplies these elements. It organizes task-local

context, tools, collaborating agents, memory access, workflow, permissions, evaluation, and submission conditions around the base model.

Seen from search, the harness performs two complementary operations. Tools and interfaces add executable edges that the model could not otherwise take. State representations, types, permissions, and workflows remove invalid or unsafe branches. Candidate generation, evaluation, and stopping rules then direct limited computation within the remaining graph. SWE-agent [15] demonstrates that changing the agent–computer interface can materially alter coding-agent behavior and performance.

Reachability is only part of the story. Two harnesses may expose the same terminal candidates but induce different state-visitation distributions under limited computation. The reachable set says what can in principle be searched. The visitation distribution says what is actually searched. A harness determines both.

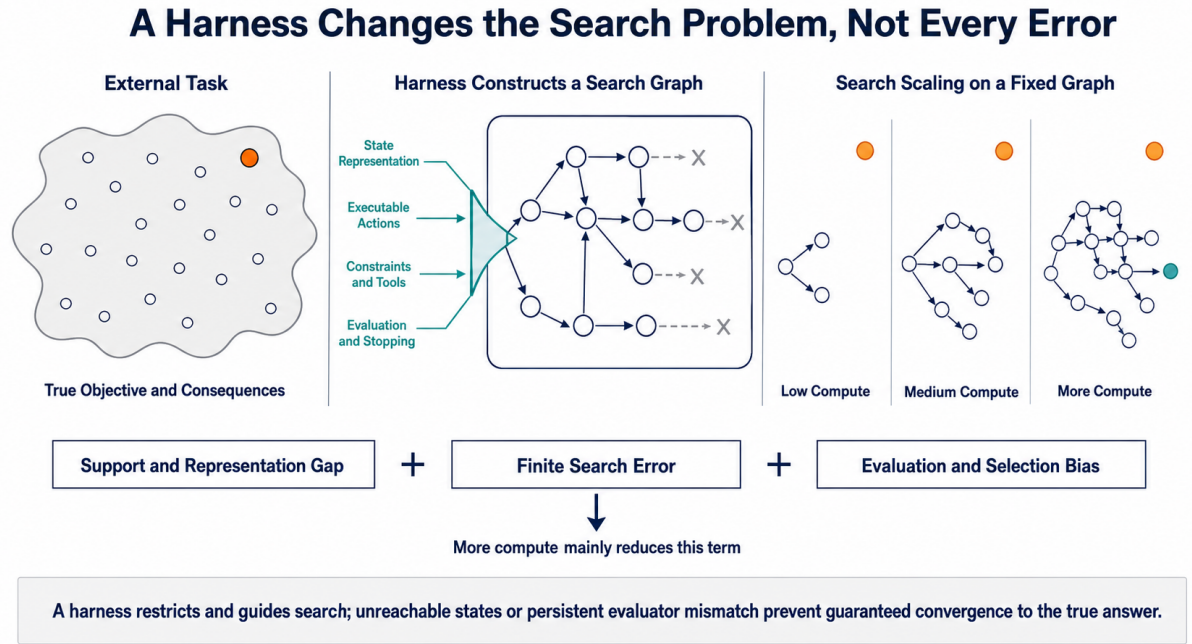


Figure 3. State representation, executable actions, constraints, tools, and evaluation define an internal search graph. More task-time computation mainly reduces finite-search error inside that graph; missing support or state and persistent evaluator mismatch require separate correction.

3.3 Three residual gaps in fixed-model search

The error of fixed-model search can be usefully separated into three parts.

Support and representation gap. A necessary state distinction is missing, an action cannot be expressed, or the candidate has effectively zero probability. More search cannot visit what the system cannot represent or reach.

Finite-search gap. A good candidate is reachable and recognizable, but limited computation fails to visit it. This is the component that additional search most directly reduces.

Evaluation and selection gap. A good candidate appears, but the system selects another because its evaluator systematically differs from real task utility. Deeper optimization of the same evaluator may exploit the discrepancy more efficiently.

In autonomous driving, a new construction pattern omitted from state, or a coordination maneuver absent from the action interface, creates a support gap. A safe trajectory that exists

but is not expanded creates a finite-search gap. A safe trajectory that appears but loses because short-term traffic flow is overvalued relative to later risk creates an evaluation gap.

The same three gaps admit a formal bias–variance description. For a fixed task x , let $\mathcal{Z}_{\text{real}}(x)$ be the candidates executable in the external environment and let $V(z; x)$ be the expected value of candidate z under the true long-run objective. With learning state and harness fixed, the system searches only

$$\mathcal{Z}_{\Lambda, \mathcal{H}}(x) \subseteq \mathcal{Z}_{\text{real}}(x)$$

and ranks candidates with an internal evaluation $\widehat{V}_{\Lambda, \mathcal{H}}(z; x)$ formed from its learned consequence model, value estimate, or evaluator. Define

$$z^* \in \arg \max_{z \in \mathcal{Z}_{\text{real}}(x)} V(z; x), \quad z_{\mathcal{H}}^* \in \arg \max_{z \in \mathcal{Z}_{\Lambda, \mathcal{H}}(x)} V(z; x),$$

and the optimum of the internally formulated search problem,

$$\tilde{z}_{\mathcal{H}} \in \arg \max_{z \in \mathcal{Z}_{\Lambda, \mathcal{H}}(x)} \widehat{V}_{\Lambda, \mathcal{H}}(z; x).$$

If the search algorithm is consistent for this fixed internal problem, its submitted result approaches $\tilde{z}_{\mathcal{H}}$ as computation grows. It does not automatically approach the externally optimal candidate z^* . The asymptotic systematic gap is

$$\begin{aligned} b_{\infty}(x) &= V(z^*; x) - V(\tilde{z}_{\mathcal{H}}; x) \\ &= \underbrace{V(z^*; x) - V(z_{\mathcal{H}}^*; x)}_{\text{support and representation gap}} + \underbrace{V(z_{\mathcal{H}}^*; x) - V(\tilde{z}_{\mathcal{H}}; x)}_{\text{model, evaluation, and selection gap}}. \end{aligned}$$

The first term arises when state representation, action interfaces, or proposal support make the externally optimal behavior unreachable. The second arises when consequence prediction, value estimation, or evaluator ranking differs from true long-run value. More computation on the same internal problem cannot generally eliminate either term.

Let $\hat{z}_{c, \mathcal{H}}$ be the random submitted candidate under finite computation c . Its finite-search error under the internal evaluation is

$$\epsilon_{\text{fin}}(c; x) = \widehat{V}(\tilde{z}_{\mathcal{H}}; x) - \mathbb{E}_{\omega} [\widehat{V}(\hat{z}_{c, \mathcal{H}}; x)] \geq 0.$$

This term asks whether high-scoring candidates already expressible by the internal problem remain unvisited or unselected under finite expansion. Even when ϵ_{fin} falls with computation, true long-run value need not rise monotonically if the internal evaluation is misspecified.

Now let $Y_{c, \mathcal{H}} = V(\hat{z}_{c, \mathcal{H}}; x)$. Across independent repetitions of the same task with the same learning state and harness, define

$$b(c; x) = V(z^*; x) - \mathbb{E}_{\omega}[Y_{c, \mathcal{H}}], \quad \sigma^2(c; x) = \text{Var}_{\omega}[Y_{c, \mathcal{H}}].$$

The standard decomposition is

$$\mathbb{E}_{\omega} [(Y_{c, \mathcal{H}} - V(z^*; x))^2] = b(c; x)^2 + \sigma^2(c; x).$$

This variance is conditional run-to-run variation on one task, induced by token or action sampling, stochastic tool returns, environment simulation, and branch ordering. Variation in task difficulty is a separate between-task component and should not be folded into the same

quantity. Additional computation can reduce finite-search error and run-to-run variation for some search procedures, but not as an unconditional property of arbitrary search.

A harness moves all of these quantities. Pruning, type constraints, and concentrated proposals often reduce branching and finite-search variation, but may enlarge the support and representation gap. Tools and executors can expand the reachable set, but may also introduce additional branches and stochastic feedback. Better environmental feedback and evaluators can reduce model, evaluation, and selection gaps; a small systematic error in a fixed proxy objective can instead be exploited more consistently by deeper search. Harness design therefore expresses a precise bias–variance trade-off, not a general rule that a smaller search space is better. It can change the entire conditional curve without guaranteeing asymptotic unbiasedness.

3.4 Why more search is not automatically MCTS-like convergence

MCTS consistency results rely on explicit conditions: a well-defined state and action process, usable environment transitions or a faithful simulator, bounded rewards, and sufficient exploration. Open-ended LLM systems usually satisfy these conditions only approximately. Their candidate distribution is learned from historical data; internal state is a partial representation of the external task; tool feedback may be sparse or delayed; and final evaluation may itself be a learned approximation.

More candidates can reduce sampling error under a fixed generator. They do not automatically remove missing support or align a fixed evaluator with real utility. Search actively finds candidates favored by its evaluation procedure. If that procedure contains a systematic error, deeper search may either find a genuinely better solution or exploit the error more effectively.

The limit of a Search Scaling curve is therefore conditional on the model, harness, representation, reachable action set, candidate generator, and evaluator that define the curve. Pretraining can improve all of these starting points. It is not a consistency theorem for arbitrary real-world optimization.

3.5 Why offline model iteration is still insufficient

Frozen models and periodic releases remain powerful engineering choices. They provide reproducibility, centralized safety review, and the ability to aggregate shared structure. They also have four persistent limitations:

1. environmental change occurs between releases, creating adaptation delay;
2. personal, organizational, and private regularities may not be suitable for centralized training;
3. future observations depend on the deployed policy, whereas offline records were generated by older policies;
4. repeated structure is solved again from the same frozen candidate and value prior, paying recurring search and verification cost.

Runtime continual learning can update part of the state that shapes later search: predictive representation, candidate prior, value estimate, memory, or harness. Such learning may expand support, direct computation better, or correct evaluation. This is how learning can change a future Search Scaling curve. But Learning Scaling is not defined by curve movement alone. Its object remains long-run control, recovery, transfer, retention, plasticity, and later learning efficiency.

4 What Learning Scaling must study

Many systems called “continual learning” optimize the wrong object. They count updates, stored memories, remembered benchmark items, or parameter change. These quantities describe mechanisms or diagnostics. Continual reinforcement learning begins with a continuing agent–environment process and an objective over that process. MemRL [16], for example, applies runtime reinforcement learning to episodic memory and therefore offers a concrete non-parametric route to experience reuse and runtime improvement. It should still be distinguished from the stronger Alberta-style formulation in which task parameters, meta-parameters, and feature construction co-evolve.

4.1 Experience is first-person and action-conditioned

Let a history contain the agent’s observations, actions, rewards, and other feedback. The crucial fact is causal, not terminological: actions alter the observations that follow. Two learning rules beginning from the same initial distribution can induce different streams of experience because they choose different actions. A recorded dataset can be replayed to both rules, but replay removes this policy–experience coupling.

This is why “more deployment data” is not a sufficient definition of Learning Scaling. A central trainer may aggregate logs from a population. A deployed agent, by contrast, learns along the particular history produced by its own decisions in its own environment.

The Alberta Plan [17] treats the agent as a resource-limited system that continually perceives, acts, and learns. The Era of Experience [6] similarly emphasizes long streams of actions and environmental consequences. This orientation changes the unit of analysis from a static dataset–model pair to a continuing control process.

4.2 State must be constructed from history

The environment does not hand an agent a complete and stationary Markov state. The same current observation can arise from different histories and require different actions. A continuing agent must recursively construct an internal state that preserves distinctions useful for prediction and control.

Larger context or memory may retain more history without turning it into a decision-relevant state. The system still needs to learn which distinctions matter, how actions affect later outcomes, and how the state representation itself should change. Persistent memory is one possible carrier of learning state; it is not the objective of learning.

Continual reinforcement learning also requires the ability to form abstractions from experience. Abstraction here does not mean merely assigning a broader linguistic label. It means learning which distinctions in history matter for prediction and control, which action sequences can function as temporally extended behaviors, and over which time scales consequences should be predicted. State abstraction determines the internal variables on which prediction and decision depend; temporal abstraction [18] organizes behavior and credit assignment beyond individual time steps. If these abstractions are fixed entirely by the designer, the agent can update only within an inherited problem formulation. It lacks a mechanism for changing that formulation when the environment exposes previously unrepresented structure.

Sutton and Javed [5] identify a missing capability in current systems: forming new abstractions from the agent’s own experience, learning predictions in terms of them, and planning with them. Useful abstractions cannot be specified once for all environments; the agent must discover those appropriate to the world it actually encounters. Abstraction is therefore not in-

dependent of experience: feature construction, state construction, and multi-timescale behavior representations must remain open to empirical revision during a long interaction history.

4.3 A continuing learner must also construct what to learn next

It is important to distinguish a **long-run control objective** from an **intermediate learning target**. The former specifies the task utility, terminal criterion, or safety constraints by which the agent is ultimately evaluated. The latter selects which opponent, question, prediction error, behavioral distinction, or environmental variation should produce the next informative segment of experience. Continual learning should not freely rewrite the long-run objective, but it can construct, test, and revise intermediate learning targets from its history and current capability.

AlphaZero [3] provides a clean boundary case. The game rules and win-loss criterion are externally fixed; self-play does not decide what winning means. It endogenously generates opponents, positions, trajectories, and a training distribution whose difficulty tracks the current learner. Unlike expanding the search tree in one position, self-play changes the experience distribution used for later policy and value updates.

Self-play under fixed rules is only one form of curriculum construction. Learning to Design Games [19] also optimizes an environment designer so that interaction produces challenging training problems. Diverse Auto-Curriculum [20] shows why increasing challenge alone is insufficient for non-transitive and diverse multi-agent interaction. Neural Auto-Curricula [21] uses meta-gradients to learn both opponent selection and the response update. Language Games [22] extends the same problem to role fluidity, reward variety, and rule plasticity, allowing language interaction to generate new learning problems. It is a research position paper rather than a generally validated continual-RL algorithm.

This yields another core distinction between the two scaling relations. With the long-run objective, problem formulation, and learning-target generator fixed, Search Scaling expands and compares more candidates for the present problem. Learning Scaling may persistently improve how experience selects future predictions, opponents, behavioral distinctions, or environmental variations as learning targets—and must show that these changes improve externally evaluated control and transfer. Search solves an already formulated problem more thoroughly; learning can also improve the process that formulates subsequent learning problems.

Self-generated learning targets are not automatically valid. A fixed game supplies an unambiguous terminal criterion; open language and social interaction can contain conflicting or cyclic rewards, roles, and rules, or allow exploitation of a current evaluator. Endogenous targets therefore remain constrained by an externally specified long-run objective, safety conditions, and independent evaluation. Otherwise a system may become better at generating internally coherent tasks without becoming a better continuing learner in the world.

Reward Is Enough [23] proposes that, in a sufficiently rich environment, multiple intelligent abilities may arise in the service of maximizing cumulative reward. This is a hypothesis about a unified objective. In the framework used here, long-run reward specifies the control objective that behavior ultimately serves; opponent selection, auxiliary prediction, exploration questions, and curriculum generation are intermediate learning structures for pursuing that objective. Learning Scaling asks whether those structures improve through experience. It does not require the agent to rewrite its long-run objective without constraint. AlphaZero [3] shows the resulting search-learning cycle: tree search improves present action selection, self-play generates experience, and learning changes the policy and value estimates used by subsequent search.

Taken together, the Sutton-Silver research line does not motivate a transition from search to learning, still less the replacement of planning by continual learning. *The Bitter Lesson* treats search and learning as general methods that can continue to exploit computation. *The Alberta Plan* places the agent in a continuing process that updates prediction, control, representation,

and meta-parameters through time. *The Era of Experience* centers capability formation on long, action-conditioned streams with environmental consequences. Learning Scaling adds a separate empirical question: whether such experience improves the starting point of later search and learning, and whether the improvement persists in continuing control.

4.4 The objective is continuing control, not continuous parameter motion

A long-run objective must be declared independently of the implementation. Depending on the domain, this may be average reward, discounted return, constrained utility, or a task-specific control criterion. Environment changes make recovery and opportunity cost part of the objective: a system that eventually readapts after losing months of reward is different from one that tracks change with little loss.

Continual learning research often uses task sequences because they make transfer, forgetting, and plasticity easier to diagnose. These are valuable diagnostics. They do not replace actual control on the policy-induced stream. A system may improve on future held-out tasks while degrading the return obtained during the experience that supposedly taught it.

4.5 Alberta-style Learning Scaling changes the learner

Explicit search and planning usually operate in a current task-structure space. State representation, proposal distribution, value approximation, and update rules remain fixed within an independent run; additional computation expands more nodes, trajectories, or program combinations. Candidate archives, long-term memory, and retrieval can enlarge the historical content available to later runs. If they only enlarge what can be retrieved, without changing state construction, credit assignment, or how new experience updates the agent, they are not equivalent to evolution of the learning process itself.

The Alberta Plan [17] proposes a deeper route. Temporal uniformity asks perception, prediction, control, planning, and learning to continue under the same principles at every time step rather than relying on a privileged training phase followed by a permanently frozen learner. Its research sequence begins with per-feature and per-weight step-size meta-learning, then moves through generate-test-replace feature discovery, history-dependent state construction, and continual actor-critic control. The important synthesis is not one algorithm name but three coupled levels:

1. **Task parameters** determine current prediction, value, and policy.
2. **Meta-parameters** regulate per-parameter step sizes, normalization, and update rates so stable knowledge and new structure can operate on different time constants.
3. **Feature, state, and abstraction construction** generate, test, and replace internal representations and learn state and behavioral abstractions at multiple time scales, changing which distinctions later prediction, evaluation, planning, and control can use.

These three internal levels must also connect to **intermediate learning-target construction**. Current parameters, representations, and meta-parameters help select which opponents, problems, and predictive distinctions are worth learning next; subsequent experience tests those choices and revises the target-generation process. If that process remains fixed, an agent may keep updating on a supplied curriculum without improving its ability to formulate future learning problems.

The policy changes what the agent observes next, so these levels do not optimize an externally fixed stream independently. They co-evolve with a first-person, action-conditioned experience process. Past experience can therefore change not only retrievable content but also the

representation and update process that will learn from future experience. This is the strong, Alberta-style form of Learning Scaling developed here.

It is stronger than merely having persistent state. It is also narrower than the essay’s general definition of possible learning carriers. Weight change is not necessary for every form of Learning Scaling; memory, reusable policy structures, or a harness may carry consequential updates. But if the aim is an agent that keeps forming new abstractions and improves how it learns during a long run, storing more records or preserving more candidates will usually be insufficient. Representation and update dynamics must remain capable of change.

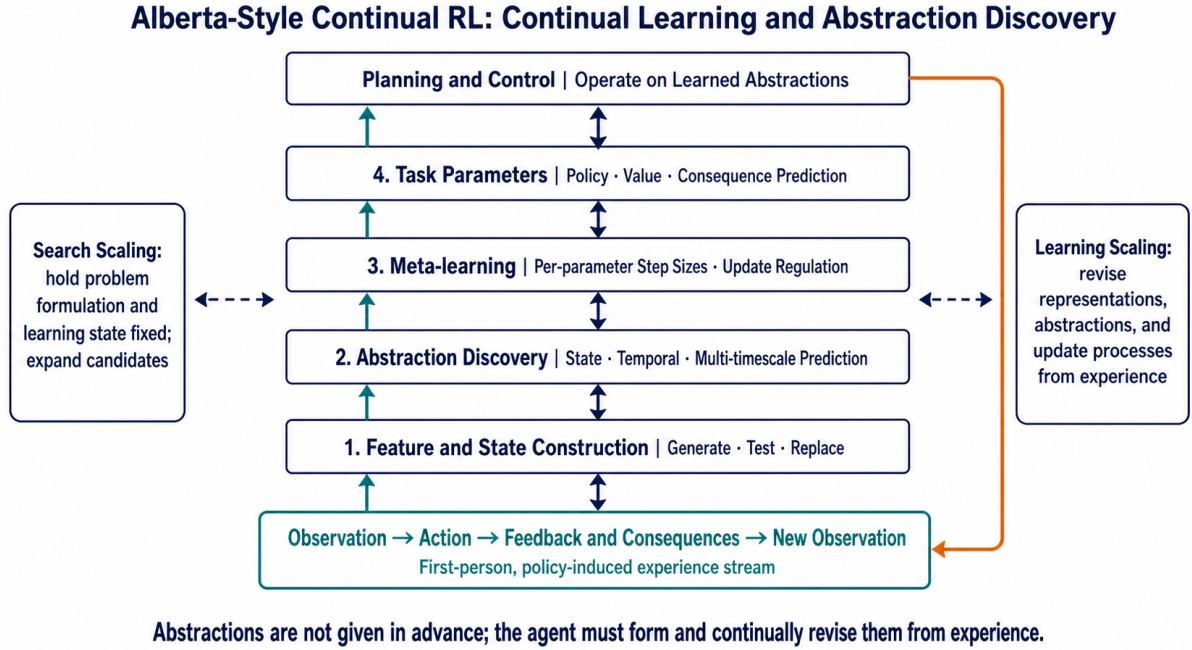


Figure 4. Search expands candidates under given parameters, meta-parameters, and a given problem formulation. Alberta-style continual RL must also form state and temporal abstractions from first-person experience and let them co-evolve with task parameters and update dynamics. The route becomes run-time RSI only if prior learning causally improves later learning efficiency; otherwise it remains continual adaptation.

4.6 Plasticity is necessary but not the final objective

Deep networks can lose the ability to learn long before they lose all previously acquired performance. Dohare et al. [24] distinguish catastrophic forgetting from loss of plasticity: a network may retain old behavior yet respond poorly to new training signals after prolonged non-stationary learning.

Their Continual ImageNet setting reports severe deterioration in later learning and examines mechanisms associated with dormant units, weight growth, and reduced effective adaptation. Continual Backprop periodically replaces low-utility features while preserving the rest of the network, a generate-and-test mechanism intended to maintain feature renewal. This is important evidence that the standard deep-learning update process is not automatically suitable for indefinitely changing targets.

The finding should not be overextended. Maintaining plasticity on supervised non-stationary streams does not by itself establish scalable continual reinforcement learning in large language-model agents. It addresses one necessary property of the update process. Long-run control, action-conditioned experience, credit assignment, retention, and stability remain separate requirements.

The emerging research direction is broader than one optimizer. Per-parameter step-size adaptation, feature renewal, online representation learning, predictive state construction, and update gating all ask how an agent can remain capable of change after extensive prior learning. Their success should ultimately be judged by whether that continuing ability improves control, not only whether gradient norms or training losses remain active.

4.7 Three curves that should not be collapsed

It is useful to distinguish three response relations:

1. **Task-time Search Scaling:** performance versus computation for one task, with persistent learning state fixed.
2. **Experience response:** later performance or continuing control versus accumulated experience, with future computation fixed.
3. **Recursive learning response:** new-task learning curves at progressively later points in a long run, asking whether the agent remains as learnable—or becomes more learnable—after more prior experience.

The second is a necessary component of Learning Scaling. The third diagnoses whether the learning process can sustain improvement rather than merely accumulate a better starting point. An agent may begin future tasks at a higher level yet require just as much new experience to master unfamiliar structure. Conversely, it may preserve fast adaptation even when the immediate starting score changes little.

Abstraction discovery and learning-target construction are additional diagnostics, not separate scaling relations. An abstraction should improve prediction, planning, or control under structural variation; support transfer beyond nearly repeated histories; and remain revisable when later experience contradicts it. A target generator should select opponents, subproblems, predictions, or environment variations that produce informative experience, improve externally evaluated control and transfer, and stop prioritizing targets once they cease to be useful. Representational clustering, a human-readable label, or a growing self-generated task set alone does not establish either capability.

5 From persistent updates to improvement dynamics

The labels self-evolving, AI4AI, and recursive self-improvement are often placed on a single ladder. They answer different questions.

Self-evolving describes how much of an update process is executed autonomously. The update may affect prompts, memory, a harness, code, data mixtures, rewards, or model parameters. Autonomous execution says nothing by itself about whether the update is useful or whether learning accelerates.

AI4AI describes the object of improvement. A system that generates, evaluates, and selects better AI code, algorithms, environments, harnesses, or training procedures belongs here. It may work through design search without a single deployed agent learning continuously.

Recursive self-improvement (RSI) describes a cross-round dynamic: current capability causally increases the rate of later capability improvement. One successful self-edit, one stronger descendant, or many autonomous rounds is not sufficient. Establishing this relation requires multiple rounds, comparable external inputs, and an estimate of whether stronger systems improve the productivity of the next improvement round.

The most mature public mechanisms in current AI4AI and self-evolving systems are usually systems composition and design search. A language model proposes code, harnesses, algo-

rithms, or training procedures; an external executor runs candidates; an automated evaluator ranks them; an archive retains selected designs; and descendants enter another round. AlphaEvolve [25] and the Darwin Gödel Machine [26] clearly instantiate this route; ML-Master [27] combines parallel exploration trajectories with selectively scoped memory for machine-learning engineering search. These are meaningful accomplishments in executable design spaces. They primarily enlarge AI-research search, however. A growing archive, a changed systems composition, or stronger descendants do not establish that one continuing agent has learned to regulate its own parameters, meta-parameters, and feature construction.

The broad AI4AI label can therefore conceal a difficult mechanistic divide. One route composes existing models, tools, evaluators, and engineering components into a more effective outer search. The other keeps a learner plastic on a non-stationary, action-conditioned stream and makes earlier learning improve later learning. Design search can discover the latter, and Alberta-style continual learning can be incorporated into an AI4AI system. Until continuing-interaction studies establish that process, systems iteration should not be presented as a solved substrate-level continual-learning problem.

The form of agency at stake is developmental, not merely operational. An agent acts and thereby shapes its experience; experience updates parameters and representation; meta-learning regulates later updates; new internal structure changes what the agent can learn next. This essay treats that process as a foundational hypothesis for long-horizon agency, not as a property already established in current general-purpose agents.

5.1 Route A: AI research search

Systems such as AlphaEvolve and the Darwin Gödel Machine generate candidate programs or agents, evaluate them, keep an archive, and use selected results in later design rounds. Their immediate independent variable is design-search computation or design iteration. Current capability may improve later proposal quality, evaluation, experiment throughput, and submission quality.

This route can produce powerful feedback without one persistent deployed identity. It is search over AI designs. A stronger candidate entering the next round creates an iterated search process; RSI additionally requires stronger candidates to make subsequent improvement faster under comparable external resources.

5.2 Route B: runtime continual learning

A continuing agent may improve state construction, consequence prediction, temporal credit assignment, representation learning, behavior abstraction, step-size control, or update gating. If later unfamiliar tasks require less experience, the agent exhibits a recursive learning response. If this higher learning efficiency raises the subsequent rate of capability improvement, runtime learning participates in RSI.

The two routes may interact. AI research systems can design better continual-learning rules; continuing agents can generate more informative research experience. They remain empirically distinguishable because their state, independent variable, and unit of recurrence differ.

Two Routes to Recursive Improvement

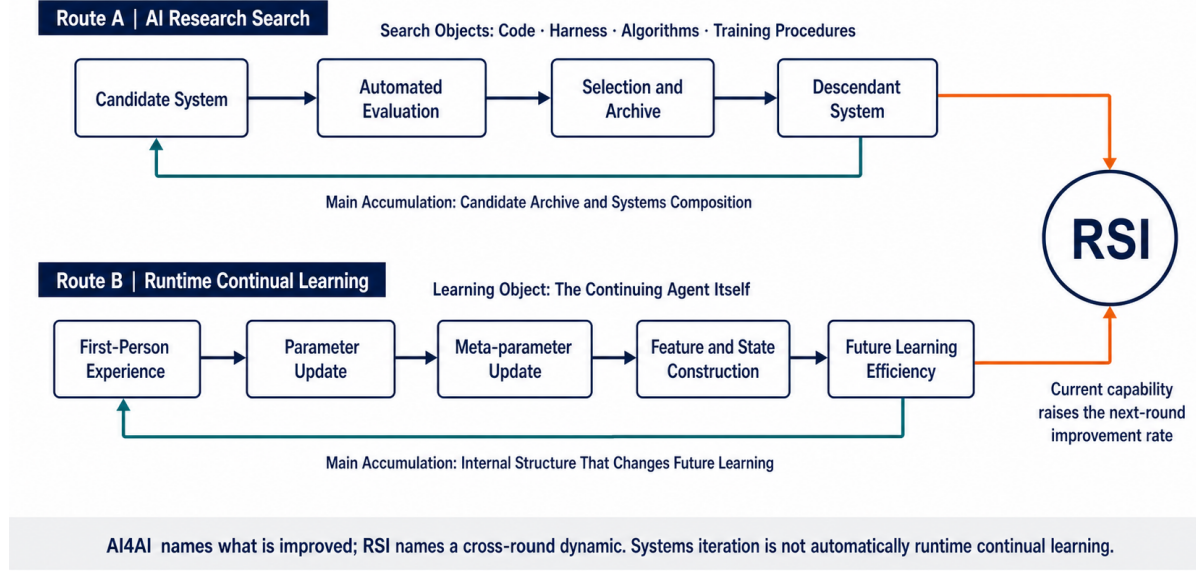


Figure 5. AI research search mainly accumulates candidate archives and systems compositions. Runtime continual learning changes parameters, meta-parameters, and feature construction along one agent’s experience history. Either route can enter RSI, but only if current capability raises the next-round improvement rate.

6 How to evaluate Search Scaling and Learning Scaling

Any scaling analysis needs an independent variable, a system specification that remains fixed, an evaluation distribution, and more than two resource levels. Comparing “low” and “high” computation, or before and after training, establishes a local difference. It does not determine the functional form of a response or establish a scaling law.

6.1 Evaluating Search Scaling

Begin from the same initial learning state. Hold the model, harness, task distribution, candidate generator, evaluator, compute-allocation rule, stopping rule, and submission rule fixed. Vary task-time computation over several levels.

The primary response may be task return, success rate, constraint violation, or submission quality. Mechanistic diagnostics should track the three gaps introduced earlier:

- coverage of high-quality candidates within the reachable set;
- effective candidate diversity and state visitation under finite computation;
- evaluator calibration, ranking accuracy, and selection error.

These quantities explain whether a curve improves because more useful candidates are visited, plateaus because support is exhausted, or declines because a fixed evaluator is increasingly exploited. Changing the harness is not another point on the same curve. It defines another conditional curve, and the curves may cross at different compute levels.

The formalization also requires systematic discrepancy to be separated from run-to-run variation. When an external task-value reference is available, repeated runs of the same task with the same learning state and harness can estimate $b(c; x)$ and $\sigma^2(c; x)$ at each computation level; aggregation across tasks is a separate description of task composition. When externally optimal

value is unavailable, the system should not be declared unbiased. Reachable-candidate coverage, consequence-prediction error, evaluator calibration, and selection error instead describe the observable parts of the discrepancy.

6.2 Evaluating Learning Scaling

State the continuing control objective first. Compare behavior at multiple levels of accumulated first-person experience while keeping future search computation, tool access, and evaluation procedures comparable.

Formally, every accumulated-experience level e induces a learning state Λ_e through actual interaction, and evaluation reads $F_L(e \mid c^*)$ at a common future computation level c^* . If experience and future computation increase together, only their joint change is observed; the entire difference cannot be attributed to Learning Scaling.

Two views are needed. **Continuing-run evaluation** preserves the relation between action and later experience. It reports long-run return or reward rate, constraint violations, recovery after environmental change, and dynamic regret relative to a stated time-varying feasible reference. **New-task diagnostics** clear task-local context, search trees, and temporary caches, then examine transfer, retention, late plasticity, and the new experience required to reach a common performance level.

Evaluation should also examine learning-target construction directly. At later stages of a run, do selected opponents, subproblems, predictive questions, or environment variations produce more informative experience? Do they improve control and transfer under an external evaluation process rather than only the learner's own score? Does the system revise or abandon a target once its marginal learning value declines? These tests separate an improving learning process from an expanding archive of self-generated tasks.

Recorded-experience replay has a narrower role. Giving two update rules the same observation–action–feedback sequence can isolate how they use a given record. It also turns experience into an external input and removes each policy's effect on what happens next. Replay can diagnose an update rule; it cannot replace continuing control.

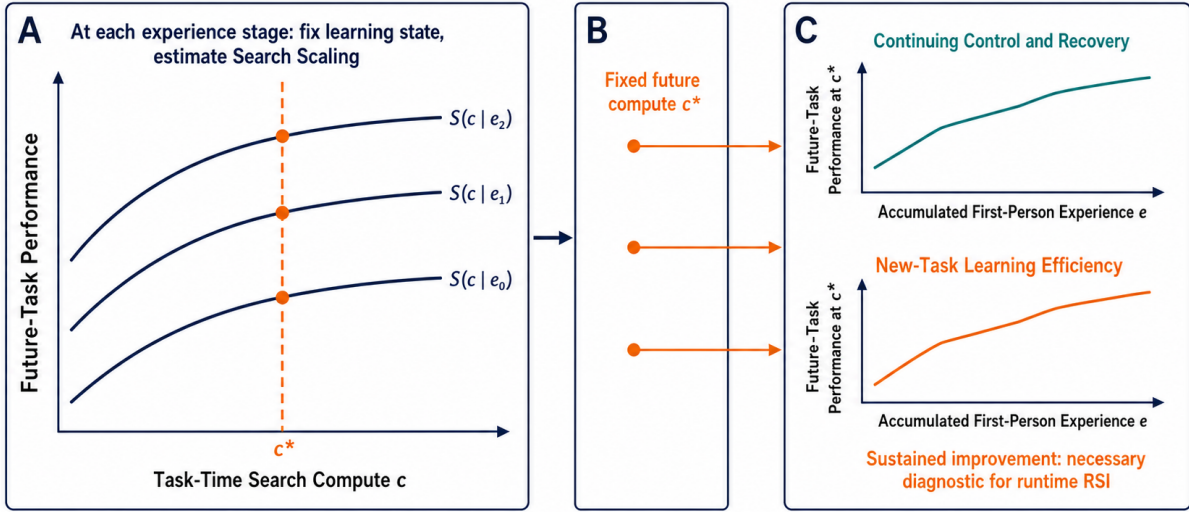
6.3 Evaluating the interaction

Place the two independent variables on a grid: task-time search computation on one axis, and accumulated experience incorporated through persistent updates on the other. At each experience checkpoint, estimate a Search Scaling curve. This reveals whether learning improves candidate generation, evaluation, and compute allocation; moves a plateau; or creates negative interaction through false memory, reduced diversity, or evaluator mismatch.

The reverse interaction matters equally. Search determines which actions are tried, so it shapes the experience from which the agent can learn. A system can spend additional task-time computation not only on exploitation but also on actions that are informative for future learning. The value of such search belongs to a long-run control objective, not only to the current task score.

An Alberta-style evaluation should additionally report which level of the learner changes. A retrieval-only system should be compared with a fixed-feature system that updates task parameters; per-parameter step-size or other meta-parameter updates should be compared with fixed step sizes or a conventional optimizer; feature generate-test-replace and state construction should be compared with a fixed feature space. All comparisons keep future task-time computation equal and examine continuing control, new-task learning efficiency, and late plasticity rather than parameter displacement or stored-item count alone. Concluding that the learner itself evolves requires the coupled levels to improve later learning.

Jointly Evaluating Two Scaling Relations



Estimate search curves by stage, then compare learning at the same future compute.

Figure 6. The left panel estimates a Search Scaling curve at each accumulated-experience stage. The middle panel reads every curve at the same future computation c^* . The right panel then assesses continuing control and new-task learning efficiency against accumulated first-person experience. The first relation evaluates Learning Scaling; sustained improvement in the second is a necessary diagnostic for runtime RSI.

The practical allocation is conditional. Rare and highly compositional problems may remain best handled by task-time search. Repeated regularities with transferable structure are candidates for persistent learning. Periodic offline training may dominate when centralized governance and stability matter most. Runtime learning becomes more valuable as the environment changes faster, relevant experience becomes more local, and repeated search costs grow.

Search and learning therefore do not form a value ranking. AlphaZero is the clearest counterexample to any forced opposition: learning forms policy and value estimates; search uses them to allocate node expansion; self-play generates new experience; learning changes the next search. The scientific task is to state which relation is being varied and which conditions remain fixed.

7 Conclusion

Recorded real experience describes interactions that have already occurred. Synthetic data extend conditions that designers can generate, simulate, and verify. Pretraining and RL post-training convert both into representations, candidate generation, action-consequence prediction, policies, and approximate values. Together they produce a strong deployment prior and make a large language model a remarkably general source of search guidance.

That prior remains an approximation. Collection coverage, synthetic generators, training objectives, reward processes, and future deployment distributions do not coincide perfectly. A harness can formulate the task better, add executable actions, restrict invalid branches, concentrate computation, and improve evaluation. It can also exclude a correct path or preserve a mistaken evaluator. More task-time computation primarily reduces finite-search error inside the fixed model-harness system. It does not universally remove missing support, missing state, or systematic evaluation error.

This is why the autonomous-driving example matters. If a safe trajectory is reachable and recognizable but missed by a shallow rollout, more search may recover it. If the road change is absent from state, the necessary coordination maneuver cannot be proposed, or the evaluator persistently discounts later risk, generating more trajectories does not automatically repair the problem.

Offline model iteration remains essential. It can aggregate common structure, apply centralized review, and release stronger shared priors. It is still separated in time from local deployment experience. Runtime continual learning asks whether an agent can use the consequences of its own actions to correct later prediction and control while remaining stable, retaining old ability, and preserving the capacity to learn new structure.

Search Scaling studies task-time computation from a fixed persistent state. Learning Scaling studies whether accumulated action-conditioned experience improves continuing control and sustained learnability under comparable future conditions. RSI studies whether current capability raises the rate of later capability improvement. They can interact; none can be inferred from another.

The title therefore marks an addition to the research agenda. Search Scaling is increasingly measurable at the output, reasoning, action, harness, and AI-design levels. The missing curve asks whether, under a stated long-run objective and experience process, a deployed agent can keep converting first-person experience into better control, recovery, transfer, retention, late plasticity, and more efficient learning of unfamiliar structure.

Persistent update is not enough. Deep learning can lose plasticity during prolonged non-stationary training, just as fixed search can preserve representation and evaluation error under increasing computation. Learning Scaling requires an update process that remains capable of useful change and serves actual long-run control.

Next essay — From Modern Deep Reinforcement Learning to Continual Reinforcement Learning

The next article will begin with episodic reinforcement learning and the training structure of modern deep RL, then compare them with large-language-model RL post-training and continual reinforcement learning. It will examine how continuing experience, long-run control, state construction, long-horizon credit assignment, tracking in non-stationary environments, and lasting plasticity change the central problem of reinforcement learning.

8 Terminology

Term	Meaning in this essay
Learning	Experience causes a persistent state update that changes later prediction, action, or further learning
Training	State is updated through specified data, objectives, optimization, and stopping conditions; pretraining, supervised fine-tuning, and RL post-training are training
Testing	Performance estimation on a declared task set, metrics, and operating conditions
Deployment	Operation for real users, tasks, or environments; deployment may use frozen inference or persistent learning

Term	Meaning in this essay
Inference	Computation from current input and existing state to a prediction, answer, or action
Reasoning	Intermediate computation used to form a judgment or action
Search	Generating, expanding, evaluating, and selecting candidates in a defined candidate space
Planning	Comparing temporally ordered actions through their future consequences; planning may use search, while search need not be planning
Agent	A system that receives observations and acts in continuing interaction, with actions affecting later observations and feedback
Runtime continual learning	Persistent learning during deployment interaction, so experience continues to affect later prediction, control, or learning
Continual reinforcement learning	Continuing learning along a first-person, action-conditioned experience stream in service of a declared long-run control objective
Harness	The system around a base model that organizes task-local context, tools, collaborating agents, workflow, permissions, memory, evaluation, and commitment

Task-time training that is cleared after the current problem is task-local adaptation. It enters runtime learning only when the update persists across tasks and affects later conditions. Search Scaling and Learning Scaling refer to conditional response relations with declared independent variables and held-fixed conditions.

References

- [1] Richard S. Sutton. The Bitter Lesson. Online essay, 2019. URL <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- [2] Ming Zhou, Jun Luo, Julian Vilella, Yaodong Yang, David Rusu, Jiayu Miao, et al. SMARTS: An Open-Source Scalable Multi-Agent RL Training School for Autonomous Driving. In *Conference on Robot Learning*, pages 264–285, 2021. URL <https://proceedings.mlr.press/v155/zhou21a.html>.
- [3] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv preprint arXiv:1712.01815, 2017. URL <https://arxiv.org/abs/1712.01815>.
- [4] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. arXiv preprint arXiv:1703.06907, 2017. URL <https://arxiv.org/abs/1703.06907>.
- [5] Sequoia Capital. Rich Sutton and Khurram Javed: Why AI Models Stop Learning, and How to Start It Again. Podcast interview with Richard Sutton and Khurram Javed, 2026. URL <https://sequoiacap.com/podcast/rich-sutton-and-khurram-javed-why-ai-models-stop-learning-and-how-to-start-it-again>.
- [6] David Silver and Richard S. Sutton. Welcome to the Era of Experience. Preprint chapter, 2025. URL <https://storage.googleapis.com/deepmind-media/Era-of-Experience%20The%20Era%20of%20Experience%20Paper.pdf>.
- [7] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. AI models collapse when trained on recursively generated data. *Nature*, 631(8022):755–759, 2024. doi: 10.1038/s41586-024-07566-y. URL <https://www.nature.com/articles/s41586-024-07566-y>.

- [8] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, et al. Scaling Laws for Neural Language Models. arXiv preprint arXiv:2001.08361, 2020. URL <https://arxiv.org/abs/2001.08361>.
- [9] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, et al. Training Compute-Optimal Large Language Models. arXiv preprint arXiv:2203.15556, 2022. URL <https://arxiv.org/abs/2203.15556>.
- [10] Muning Wen, Ziyu Wan, Jun Wang, Weinan Zhang, and Ying Wen. Reinforcing LLM Agents via Policy Optimization with Action Decomposition. *Advances in Neural Information Processing Systems*, 37:103774–103805, 2024. doi: 10.52202/079017-3297. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/bc09efb501c801ed92e181e26a885c2d-Abstract-Conference.html.
- [11] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv preprint arXiv:2407.21787, 2024. URL <https://arxiv.org/abs/2407.21787>.
- [12] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv preprint arXiv:2408.03314, 2024. URL <https://arxiv.org/abs/2408.03314>.
- [13] Ziyu Wan, Xidong Feng, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. AlphaZero-Like Tree-Search can Guide Large Language Model Decoding and Training. In *International Conference on Machine Learning*, pages 49890–49920, 2024. URL <https://proceedings.mlr.press/v235/wan24c.html>.
- [14] Jianghao Lin, Yuanyuan Shi, Xin Peng, Renjie Ding, Hairui Wang, Yuxuan Peng, et al. ToolPRM: Fine-Grained Inference Scaling of Structured Outputs for Function Calling. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18792–18804, 2026. doi: 10.18653/v1/2026.acl-long.855. URL <https://aclanthology.org/2026.acl-long.855/>.
- [15] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv preprint arXiv:2405.15793, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [16] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, et al. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory. arXiv preprint arXiv:2601.03192, 2026. URL <https://arxiv.org/abs/2601.03192>.
- [17] Richard S. Sutton, Michael Bowling, and Patrick M. Pilarski. The Alberta Plan for AI Research. arXiv preprint arXiv:2208.11173, 2022. URL <https://arxiv.org/abs/2208.11173>.
- [18] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1. URL [https://doi.org/10.1016/S0004-3702\(99\)00052-1](https://doi.org/10.1016/S0004-3702(99)00052-1).
- [19] Haifeng Zhang, Jun Wang, Zhiming Zhou, Weinan Zhang, Yin Wen, Yong Yu, and Wenxin Li. Learning to Design Games: Strategic Environments in Reinforcement Learning. Online publication, 2018. URL <https://www.ijcai.org/proceedings/2018/426>.
- [20] Yaodong Yang, Jun Luo, Ying Wen, Oliver Slumbers, Daniel Graves, Haitham Bou Ammar, Jun Wang, and Matthew E. Taylor. Diverse Auto-Curriculum is Critical for Successful Real-World Multiagent Learning Systems. In *Proceedings of AAMAS 2021, Blue Sky Ideas Track*, pages 51–56, 2021. URL <https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p51.pdf>.
- [21] Xidong Feng, Oliver Slumbers, Ziyu Wan, Bo Liu, Stephen McAleer, Ying Wen, Jun Wang, and Yaodong Yang. Neural Auto-Curricula in Two-Player Zero-Sum Games. *Advances in Neural Information Processing Systems*, 34:3504–3517, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/1cd73be1e256a7405516501e94e892ac-Abstract.html.
- [22] Ying Wen, Ziyu Wan, and Shao Zhang. Language Games as the Pathway to Artificial Superhuman Intelligence. arXiv preprint arXiv:2501.18924, 2025. URL <https://arxiv.org/abs/2501.18924>.
- [23] David Silver, Satinder Singh, Doina Precup, and Richard S. Sutton. Reward Is Enough. *Artificial Intelligence*, 299:103535, 2021. doi: 10.1016/j.artint.2021.103535. URL <https://doi.org/10.1016/j.artint.2021.103535>.

- artint.2021.103535.
- [24] Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mah-mood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632(8026):768–774, 2024. doi: 10.1038/s41586-024-07711-7. URL <https://www.nature.com/articles/s41586-024-07711-7>.
 - [25] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131, 2025. URL <https://arxiv.org/abs/2506.13131>.
 - [26] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents. arXiv preprint arXiv:2505.22954, 2025. URL <https://arxiv.org/abs/2505.22954>.
 - [27] Zexi Liu, Yuzhu Cai, Xinyu Zhu, Yujie Zheng, Runkun Chen, Ying Wen, et al. ML-Master: Towards AI-for-AI via Integration of Exploration and Reasoning. arXiv preprint arXiv:2506.16499, 2025. URL <https://arxiv.org/abs/2506.16499>.
 - [28] OpenAI. Learning to Reason with LLMs. Technical blog, 2024. URL <https://openai.com/index/learning-to-reason-with-llms/>.
 - [29] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv preprint arXiv:2501.12948, 2025. URL <https://arxiv.org/abs/2501.12948>.
 - [30] Leo Gao, John Schulman, and Jacob Hilton. Scaling Laws for Reward Model Overoptimization. In *International Conference on Machine Learning*, pages 10835–10866, 2023. URL <https://proceedings.mlr.press/v202/gao23h.html>.
 - [31] Levente Kocsis and Csaba Szepesvári. Bandit Based Monte-Carlo Planning. In *European Conference on Machine Learning*, pages 282–293, 2006. doi: 10.1007/11871842_29. URL https://doi.org/10.1007/11871842_29.
 - [32] David Abel, Andre Barreto, Benjamin Van Roy, Doina Precup, Hado van Hasselt, and Satinder Singh. A Definition of Continual Reinforcement Learning. *Advances in Neural Information Processing Systems*, 36:50377–50407, 2023. doi: 10.52202/075280-2192. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/9d8cf1247786d6dfeefeeb53b8b5f6d7-Abstract-Conference.html.
 - [33] Esraa Elelimy, David Szepesvari, Martha White, and Michael Bowling. Rethinking the Foundations for Continual Reinforcement Learning. *Reinforcement Learning Journal*, 6:2174–2194, 2025. URL <https://rlj.cs.umass.edu/2025/papers/Paper243.html>.
 - [34] François Chollet. On the Measure of Intelligence. arXiv preprint arXiv:1911.01547, 2019. URL <https://arxiv.org/abs/1911.01547>.
 - [35] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive Self-Improvement in AI: From Bounded Self-Refinement to Autonomous Research Loops. arXiv preprint arXiv:2607.07663, 2026. URL <https://arxiv.org/abs/2607.07663>.
 - [36] Tom Cunningham et al. The Economics of Recursive Self-Improvement. Working paper, 2026. URL <https://elasticity.institute/rsi-paper.pdf>.
 - [37] Khurram Javed and Richard S. Sutton. The Big World Hypothesis and Its Ramifications for Artificial Intelligence. Reinforcement Learning Conference workshop position paper, 2024. URL https://khurramjaved.com/papers/the_big_world_hypothesis.pdf.